

LISTA DE EXERCÍCIOS 4.5

1. Defina arquivo.

Resposta: arquivo é uma unidade de armazenamento lógico a qual contém uma coleção de informações relacionadas.

2. Defina sistema de arquivo.

Resposta: é uma estrutura para a representação e mapeamento de arquivos que residem no armazenamento secundário (discos). Oferece interface de usuário para o armazenamento e mapeamento da representação lógica do arquivo para sua contraparte física. Além de acesso eficiente e conveniente ao disco permitindo que dados sejam armazenados, localizados e recuperados facilmente.

3. Destaque a importância do FS em camadas.

Resposta: no contexto de FS, camadas são úteis para a redução de complexidade e redundância na implementação das funções pertinentes para seu funcionamento (i.e. camadas de aplicação, FS lógico, módulo de organização de arquivo, FS básico, controle de I/O, dispositivos).

4. Considerando a interação com arquivos a partir de espaço de usuário, por que é vital o mapeamento do arquivo para memória, ao invés de realizar as operações de I/O apenas via interação com o armazenamento secundário?

Resposta: considerando a existência de discos mecânicos, operações de I/O representam múltiplas movimentações de cabeça de leitura/escrita, as quais são demoradas ao se considerar o tempo de operação da CPU. Esse impacto normalmente é minimizado ao se mapear o arquivo (ou parte dele) para a memória e fazendo operações de leitura/escrita sobre o arquivo aberto e mapeado em memória principal (a qual possui um tempo de acesso mais próximo à CPU).

5. Qual é a importância/papel do File Control Block (FCB)?

Resposta: o FCB contém detalhes (metadados) sobre o arquivo a ser manipulado. Esses detalhes incluem: permissões, *timestamp* (tempo de criação, último acesso, modificação), propriedade, grupo, ACL, tamanho, referência aos blocos componentes do arquivo.

6. Descreva o processo de abertura/leitura de um arquivo.

Resposta: as operações sobre arquivos, devem seguir cronologicamente um procedimento para sua abertura seguido de operações sobre o arquivo (ex. leitura/escrita). O procedimento de abertura e leitura envolvem o mapeamento/criação de estruturas em memória principal e armazenamento secundário. Ao se solicitar a abertura de um arquivo a partir de uma chamada de sistema, faz-se uma verificação inicial sobre a existência desse arquivo em uma dada estrutura de diretório e, caso exista, recupera-se o seu FCB. De posse do FCB, verifica se o arquivo já está aberto. Caso esteja, passa ao processo chamador apenas a referência à entrada na tabela de arquivos abertos, de modo que uma tabela de arquivos abertos local ao processo seja criada contendo a referência para a tabela global. Caso não esteja aberto, cria-se a entrada apropriada na tabela de arquivos abertos global e uma entrada local na tabela de arquivos abertos do processo. O procedimento de leitura segue, após a abertura do arquivo, mediante o acesso a tabela de arquivos abertos local ao processo, a qual aponta para a entrada correspondente na tabela de arquivos abertos global. Mediante a recuperação do FCB obtém-se a referência para os blocos de dados componentes do arquivo e subsequente mapeamento dos mesmo (total ou parcialmente) para a memória principal.

7. Qual é o papel do Virtual File System (VFS)?

Resposta: o Virtual File Systems (VFS), particularmente em sistemas Unix-like oferece uma forma orientada a objeto de implementar FS. Quer se dizer com isso que o VFS permite que a mesma interface de chamada de sistema (a API) seja usada para diferentes tipos de FS. Dessa forma, separa operações genéricas de FS de detalhes de implementação. A implementação pode ser uma de muitos tipos de FS, ou ainda, FS de rede. Implementa *vnodes* os quais mantêm inodes ou detalhes de arquivo em rede. Em seguida envia a operação para as rotinas de implementação apropriadas do FS. As funções do VFS são em realidade “funções de entrada” para as implementações particulares de cada FS (similar ao que se realiza com ponteiros de função).

8. Considerando método de alocação contígua e o método de alocação ligado, discorra sobre as vantagens e desvantagens de cada um deles.

Resposta: o método contíguo implementa o mapeamento do arquivo em blocos contíguos. Como vantagem tem-se potencialmente um menor tempo de acesso ao arquivo. Como desvantagem, pode ocasionar fragmentação externa. O método de alocação ligado, opera com uma representação de lista encadeada para cada bloco componente do arquivo. Como vantagem não existe fragmentação externa, porém, tem desvantagem de o tempo de acesso poder ser alto dependendo da movimentação provocada pela representação esparsa do arquivo no disco.

9. Explique brevemente o funcionamento e forneça algumas características do modelo de alocação indexado.

Resposta: segundo esse modelo de alocação de arquivos, é necessário o uso de uma tabela de índice para o acesso a blocos de arquivo em disco. Essa tabela pode ter acesso direto aos blocos em disco ou cada entrada pode fazer referência a uma ou mais tabelas, de modo a aumentar a quantidade de blocos a serem mapeados (e por conseguinte, o tamanho do arquivo a ser referenciado). O acesso aos blocos componentes do arquivo é aleatório. Não há fragmentação externa, mas possui a sobrecarga do bloco indexado

10. Discorra sobre o desempenho das estratégias de alocação discutidas nessa unidade (contíguo, ligado e indexado).

Resposta: estratégia de alocação contígua é ótima para acesso sequencial e aleatório. A estratégia de alocação ligada (linked) é boa para acesso sequencial, mas não aleatório. A estratégia indexada é mais complexa, pois o acesso ao bloco de dados pode requerer duas (ou mais) leituras de índice de bloco e então leitura de bloco de dados. O agrupamento de blocos (clustering) pode ajudar a melhorar a vazão, e reduzir sobrecarga de CPU.

11. Qual é o papel do bit vector ou bit map em gerenciamento de espaço livre?

Resposta: bit vector ou bit map permite o registro de blocos livres no FS. Com isso é possível rastrear blocos/clusters disponíveis em disco. Vale ressaltar que a estrutura de bit map requer espaço extra para armazenamento.

12. Por que a estrutura de lista ligada para representação de espaço livre em disco é eficiente?

Resposta: por que nesse caso específico, para se encontrar um espaço vazio, basta utilizar o primeiro elemento da lista ligada.

13. Por que buffer cache é uma implementação complexa em termos de integração entre gerenciamento de memória e gerenciamento de arquivos?

Resposta: o *buffer cache* é uma representação em memória de blocos frequentemente usados do arquivo. Dependendo da integração entre as unidades de gerenciamento de memória e de gerenciamento de arquivo, o *buffer cache* pode ser usado por ambas. Caso essa integração não seja adequada/otimizada, é necessário se replicar o *buffer cache* em páginas para acesso por um processo em memória principal.

14. Qual é a relação entre o *buffer cache* e os modelos de escrita síncrono e assíncrono?

Resposta: o modelo de escrita síncrono pressupõe um reconhecimento (ack) da operação em disco. Com isso, as otimizações que seriam alcançadas pelo *buffer cache* não seriam aproveitadas. No caso do modelo assíncrono, ocorre a solicitação de leitura/escrita e o processo pode realizar outras tarefas. Assim, após a bufferização do arquivo (ou parte dele) para a memória, as operações de leitura/escrita podem ocorrer independente do ack da operação em disco, sendo realizado apenas um *commit* ao final.